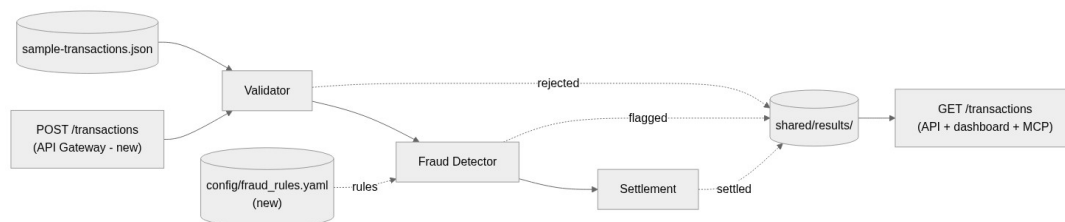


Capstone Challenge Extension

Transaction Processing Pipeline — Homework 6 Capstone Challenge
Created by Illia Chantsov

What This Extension Adds

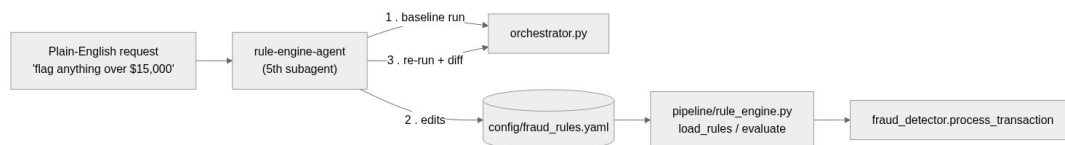
- Built on top of the already-submitted homework-6 baseline (spec, 3-stage pipeline, 4 agents, MCP, dashboard) — additive, not a revision.
- New Agent: fraud-detection scoring moves from hardcoded constants to a data-driven rule engine, editable by a 5th subagent from plain English.
- API Gateway: a write-capable REST service (api/server.py) submits transactions over HTTP, reusing the same tested pipeline stage functions the batch path uses.
- The known-outcome baseline (3 settled / 3 flagged / 2 rejected across the 8 sample transactions) is unchanged after the refactor — verified by re-running the pipeline.



Both new entry points (API gateway, config-driven rule engine) feed the same 3-stage pipeline and the same shared/results/ audit trail.

Rule Engine — Policy as Data, Not Code

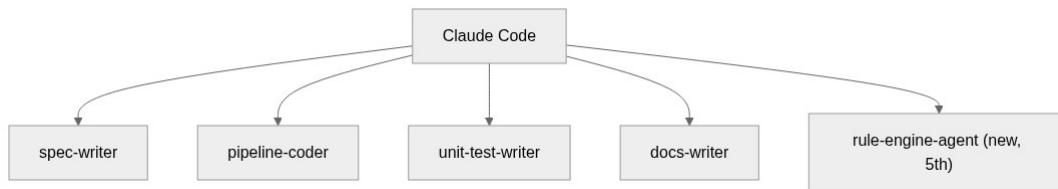
- `config/fraud_rules.yaml`: `flag_threshold` plus a list of {name, field, operator, value, score} rules. Defaults reproduce the original hardcoded thresholds exactly (\$10,000 high-value, cross-border != US, unusual timing outside 06:00-22:00 UTC).
- `pipeline/rule_engine.py`: pure functions `load_rules(path)` and `evaluate(record, ruleset)`. Operators: `gt`, `lt`, `eq`, `ne`, and `outside_hours`.
- `pipeline/fraud_detector.py` no longer holds any scoring logic itself — it only wires `rule_engine.evaluate(...)` into the file-queue envelope protocol.
- Behavior-preserving refactor: the existing `test_fraud_detector.py` suite passes unmodified against the new default rules file.



rule-engine-agent's 3-step process: baseline run, edit the YAML, re-run and diff against the baseline — every policy edit's blast radius is shown, never applied silently.

New Agent: rule-engine-agent

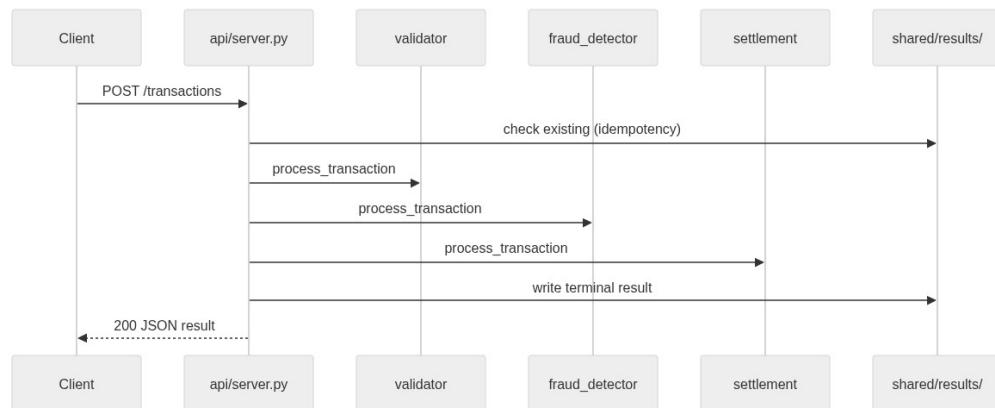
- A 5th real Claude Code subagent (.claude/agents/rule-engine-agent.agent.md), reachable via /update-fraud-rules — same pattern as the other 4 agents from the baseline.
- Tools: Read, Write, Edit, Bash — Bash specifically so it can re-run the pipeline itself, not just describe the change.
- Turns a request like "flag anything over \$15,000" into a YAML edit, then reports which of the 8 known transaction outcomes changed and confirms the rest stayed stable.
- Hard rule baked into the agent: never hand back a rules file config/fraud_rules.yaml can't parse — a failed load is treated as a hard failure to fix before reporting anything.



5 subagents total: the 4 from the baseline submission, plus rule-engine-agent.

REST API Gateway

- `api/server.py` (FastAPI) — a separate service from the read-only `frontend/server.py`, so the dashboard's read-only guarantee stays true by construction.
- `POST /transactions` runs `validator` -> `fraud_detector` -> `settlement`, same 3-stage logic the batch path uses, then writes the terminal result to `shared/results/` — an API-submitted transaction shows up in the dashboard and MCP server for free.
- `GET /transactions/{id}` and `GET /transactions` reuse the already-tested `results_reader.get_transaction / summarize_results` — no new business logic.
- Idempotent: resubmitting a known `transaction_id` returns the existing result instead of reprocessing, same rule as the orchestrator's file-queue path.



`POST /transactions` walks the same 3 pipeline stages the batch path uses, then persists to the same `shared/results/` audit trail.

Demo: API Gateway, Live

- unicorn api.server:app — a fresh transaction submitted over HTTP, scored, settled, and written to shared/results/ in one request.
- Resubmitting the same transaction_id returns the identical stored result instead of reprocessing it a second time.

```
$ curl -X POST http://127.0.0.1:8000/transactions -d '{...}'
{
  "data": {
    "transaction_id": "DEMO-API-001", "amount": "250.00",
    "status": "settled", "risk_score": 0, "flags": [],
    "fee": "1.25", "net_amount": "248.75",
    "settlement_batch": "BATCH-2026-08-13"
  }
}
```

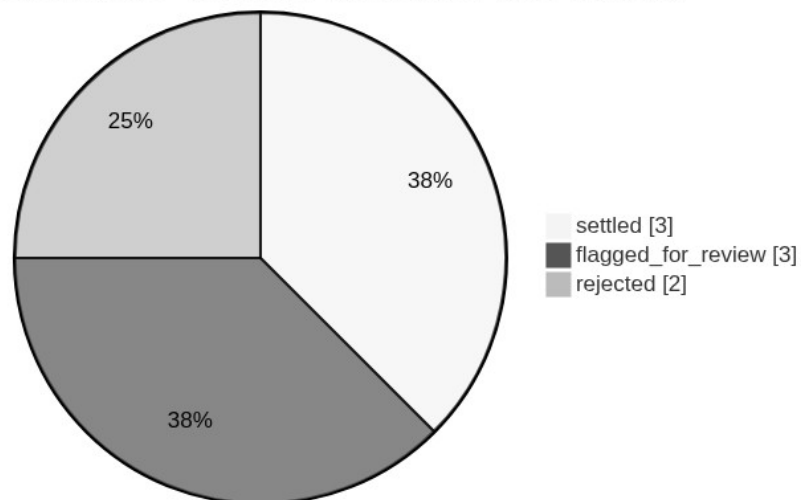
```
$ curl http://127.0.0.1:8000/health
{"status": "ok"}
```

```
$ curl -X POST http://127.0.0.1:8000/transactions -d '{"transaction_id": "DEMO-API-001"}'
# -> identical result returned, unchanged (idempotent resubmit)
```

Testing & Coverage

- 52 tests total (up from 38 in the baseline): +tests/test_rule_engine.py (default rules reproduce the exact original scores for all 8 sample transactions, plus custom rule-set edge cases) and +tests/test_api.py (FastAPI TestClient — settled / rejected / flagged submission, duplicate idempotent resubmit, 404 on an unknown transaction id).
- 93% combined coverage across pipeline + frontend + mcp + api — api/server.py and frontend/server.py both at 100%.
- Known-outcome baseline re-verified end to end after the refactor: still exactly 3 settled / 3 flagged_for_review / 2 rejected across the 8 sample transactions.

8 sample transactions - baseline unchanged after refactor



Re-ran python3 orchestrator.py after the rule-engine refactor — identical breakdown to the pre-refactor baseline.

Design Decisions

- Separate service, not a merged endpoint: `api/server.py` stays independent from `frontend/server.py` specifically so the dashboard's read-only design point remains verifiable just by reading that one file.
- No new business logic in the API layer: it calls the exact same `process_transaction` functions already covered by the baseline's stage tests — nothing to re-test.
- No authentication (matches the baseline's existing no-auth scope decision), called out explicitly as a scope choice rather than an oversight, since a write-capable endpoint is a larger exposure than the read-only dashboard.
- Rule names kept stable across edits by convention (other code/tests may reference them) — enforced by the rule-engine-agent's own instructions, not by code.